



poseo una class en c++ llamada Boton, quiero que cada instancia realice el mismo metodo, que es "leer()".

Ya esta definido

Para ejecutar el mismo método en múltiples instancias simultáneamente, la mejor práctica es agrupar los objetos en una colección (como un arreglo o un vector) e iterar sobre ellos usando un bucle.

Si estás trabajando en un **entorno de microcontroladores (como Arduino)** para prototipos físicos, es más eficiente usar un arreglo tradicional para gestionar la memoria:

C++



```
// 1. Crear un arreglo con tus instancias (ej: pasando el pin al constructor)
Boton botones[] = { Boton(2), Boton(3), Boton(4), Boton(5) };

void loop() {
    // 2. Ejecutar leer() en cada instancia usando un bucle for basado en rango
    for (auto& boton : botones) {
        boton.leer();
    }
}
```

Si estás programando en **C++ estándar** para software de escritorio, es más flexible utilizar `std::vector` :

C++



```
#include <vector>

// 1. Instanciar tus botones
Boton boton1, boton2, boton3;
```

```
// 2. Agruparlos en un vector
std::vector<Boton> botones = {boton1, boton2, boton3};

int main() {
    // 3. Iterar y ejecutar el método
    for (auto& boton : botones) {
        boton.leer();
    }

    return 0;
}
```

Dato clave: Es importante incluir el ampersand en `auto&`. Esto asegura que el bucle acceda a las instancias originales por referencia. Si solo usas `auto`, C++ creará una copia temporal de cada botón en cada ciclo del bucle, lo cual consume más memoria y puede hacer que la lectura no actualice el estado real del objeto original.

```
// esto venia en wokwi
#include <stdio.h>
#include "pico/stdlib.h"
```



Para leer todos los botones de manera conjunta sin tener que llamar a `leer()` uno por uno, la mejor estrategia es reemplazar las declaraciones individuales por un **arreglo de objetos**.

Aquí tienes tu código refactorizado. Al agrupar los botones en un arreglo, puedes usar un bucle `for` al inicio de tu `while (true)` para actualizar el estado de todos en tres líneas de código.

C++



```
// esto venia en wokwi
#include <stdio.h>
#include "pico/stdlib.h"
```

```
// incluir mis archivos
#include "Boton.h"
#include "Pote.h"
#include "Led.h"

int main() {
    stdio_init_all();

    // 1. Declarar todos los botones como un arreglo
    Boton botones[] = {
        Boton(8), Boton(9), Boton(10), Boton(11),
        Boton(12), Boton(13), Boton(14), Boton(15)
    };

    Pote miPrimerPote(26,0);

    Led ledUno(16);
    Led ledDos(17);
    Led ledTres(18);
    Led ledCuatro(19);
    Led ledCinco(20);
    Led ledSeis(21);
    Led ledSiete(22);
    Led ledOcho(27);

    while (true) {

        // 2. Leer todos los botones de manera conjunta
        for (auto& boton : botones) {
            boton.leer();
        }

        // 3. Tu lógica original, reemplazando botonUno por botones[0] y botonDos por
        if (botones[0].presionado) {
            ledUno.encenderLed();
        }

        if (ledUno.encendido == true) {
            printf("andao prendio mi loco\n");
        }

        if (botones[1].presionado) {
            ledUno.apagarLed();
        }

        if (ledUno.encendido == false) {
            printf("buuuu, que andan apagaos\n");
        }
    }
}
```

```

if (botones[0].presionado == false && botones[1].presionado == false) {
    ledDos.encenderLed();
    printf("presionen algo loco\n");
} else {
    ledDos.apagarLed();
}

sleep_ms(10);

//actualiza la lectura
miPrimerPote.leerPote();

printf("Lectura ADC: %d\n", miPrimerPote.valorPote);
printf("\n");
sleep_ms(1000);
}
}

```

Nota: Como estás trabajando con interfaces físicas (por la cantidad de botones y leds), te resultará muy útil aplicar esta misma técnica de arreglos a los `Led`. Eso te permitirá encender secuencias enteras usando variables matemáticas (ej: `leds[i].encenderLed()`) en lugar de escribir la lógica para cada uno manualmente.

estos arreglos no influyen en los constructores de la clase?

No, agrupar las instancias en un arreglo no cambia ni influye en cómo está definida tu clase `Boton` o su constructor en tu archivo `.h` o `.cpp`. El código de la clase se mantiene exactamente igual.

Lo único que ocurre al usar el arreglo con llaves `{}` es que le indicas a C++ que ejecute el constructor que **ya existe** en tu clase para cada elemento de la lista de forma secuencial.

Básicamente, esta línea:

```
Boton botones[] = { Boton(8), Boton(9), Boton(10) };
```

Hace exactamente el mismo trabajo detrás de escena que hacer esto por separado:

C++



```
Boton botonUno(8); // Llama al constructor  
Boton botonDos(9); // Llama al constructor  
Boton botonTres(10); // Llama al constructor
```

El único detalle técnico a considerar en C++:

Al inicializar el arreglo directamente con los valores (como en el ejemplo de arriba), todo funciona perfecto con tu constructor actual que recibe el pin.

Sin embargo, si en el futuro intentas crear el arreglo vacío para asignarle los pines más adelante, de esta forma:

C++



```
Boton botones[8]; // Error si la clase no tiene un constructor vacío
```

Ahí C++ sí te reclamará, porque intentará crear 8 botones usando un "constructor por defecto" (uno que no pide ningún parámetro, ej: `Boton()`). Mientras declares el arreglo asignando los objetos con su pin inmediatamente, no necesitas modificar tus constructores.

ahora si necesito llamar al boton que corresponde al pin 10,
como lo menciono?

Para referirte al botón conectado al pin 10, debes usar `botones[2]`.

Esto se debe a que los arreglos en C++ comienzan a contar desde el índice **0**. Según cómo declaraste tu arreglo, las posiciones se distribuyen de esta manera:

```
botones[0] corresponde al Pin 8 (tu antiguo botonUno )
botones[1] corresponde al Pin 9 (tu antiguo botonDos )
botones[2] corresponde al Pin 10 (tu antiguo botonTres )
botones[3] corresponde al Pin 11 (tu antiguo botonCuatro )
... y así sucesivamente hasta botones[7] para el pin 15.
```

Si necesitas comprobar el estado específico de ese botón fuera del bucle de lectura, lo escribes directamente con su índice:

C++



```
if (botones[2].presionado) {
    // Tu lógica para cuando se presiona el botón del pin 10
    ledTres.encenderLed();
}
```

```
// 2. Leer todos los botones de manera conjunta
for (auto& boton : botones) {
    boton.leer();
}
```

Este bloque es un **bucle for basado en rango** (introducido en C++11) y es la forma más limpia de recorrer todo el arreglo en milisegundos. Aunque los lee uno por uno secuencialmente, para el procesador de la Raspberry Pi Pico ocurre tan rápido que a nivel físico funciona como si los leyera todos al mismo tiempo.

Aquí tienes el desglose de qué hace cada pieza de ese código:

`botones` : Es la lista (arreglo) completa que creaste arriba con tus 8 pines.
`:` : Se lee como "dentro de".

`boton` : Es el nombre temporal que le damos al elemento actual mientras el bucle avanza. En la primera vuelta, `boton` representa a `botones[0]` (pin 8). En la segunda vuelta, representa a `botones[1]` (pin 9), y así sucesivamente.

`auto` : Le ahorra trabajo al programador. Le dice a C++ que detecte automáticamente el tipo de dato (el compilador sabe que es de clase `Boton`). Podrías escribir `for (Boton& boton : botones)` , pero `auto` es la convención moderna.

& (Ampersand o Referencia): Esta es la parte más crítica. El `&` le indica a C++ que acceda al botón original en la memoria, no a una copia. Si omites el `&` , el bucle creará un "botón clon" temporal en cada vuelta, lo leerá y luego lo destruirá, dejando tus botones originales sin actualizar.

Gracias a estas 3 líneas, tu código escala automáticamente. Si mañana decides conectar 16 botones y los agregas al arreglo principal, no tendrás que modificar este bucle de lectura; se adaptará solo a la nueva cantidad.

ya aplique el array en los leds, necesito que me ayudes a estructurar la siguiente funcion

al presionar boton[1], debe encenderse lucesita[1]



Para lograr este comportamiento, necesitas introducir un concepto clave en programación de hardware: **una variable de estado**.

Como el bucle `while (true)` se repite miles de veces por segundo, necesitas una forma de "recordar" qué botón fue el último en ser presionado para que la luz siga oscilando aunque el usuario ya haya soltado el botón.

Aquí tienes cómo estructurar la lógica dentro de tu `main` :

C++



```
int main() {
    stdio_init_all();
```

```
// (Tus arreglos de botones y lucesitas definidos aquí)
```

```

// 1. Crear una variable para recordar qué LED debe oscilar.
// Usamos -1 para indicar que al inicio no hay ninguno presionado.
int ledActivo = -1;

while (true) {
    // 2. Leer todos los botones
    for (auto& boton : botones) {
        boton.leer();
    }

    // 3. Revisar si se presionó algún botón para cambiar el estado
    for (int i = 0; i < 8; i++) {
        if (botones[i].presionado) {

            // Si el botón presionado es diferente al que ya está oscilando
            if (ledActivo != i) {

                // Si había un LED oscilando antes, lo apagamos
                if (ledActivo != -1) {
                    lucesita[ledActivo].apagarLed();
                }

                // Guardamos en la memoria el nuevo índice que debe oscilar
                ledActivo = i;
            }

            break; // Opcional: sale del ciclo for para evitar presionar dos a la \
        }
    }

    // 4. Ejecutar la acción continua basada en el estado actual
    if (ledActivo != -1) {
        lucesita[ledActivo].oscilarLed();
    }

    // (Actualización del potenciómetro y otros procesos)
    sleep_ms(10);
}
}

```

¿Por qué funciona esto?

1. **Separación de tareas:** Primero lee las entradas (botones), luego procesa la lógica (cambiar de índice) y al final ejecuta las salidas (oscilar).

2. **Memoria (`ledActivo`):** Al guardar el índice `i` en `ledActivo`, la Raspberry Pi Pico sabe exactamente a qué elemento del arreglo `lucesita` aplicarle el método en cada ciclo infinito, sin importar que `botones[i].presionado` haya vuelto a ser `false`.

Precaución sobre `oscilarLed()`: Para que la respuesta de los botones sea instantánea cuando decidas cambiar a otro, asegúrate de que tu método `oscilarLed()` no contenga pausas muy largas como `sleep_ms(500)`. Si el código se queda "pausado" dentro del método del LED, no podrá volver a la parte de arriba del `while` para leer los botones rápidamente.

desgloza y explica el código que generaste

Para entender este código, hay que pensar en el programa como un ciclo continuo que se repite miles de veces por segundo. La clave para que la luz siga oscilando después de soltar el botón es separar la acción en dos partes: **decidir qué hacer y hacerlo continuamente**.

Aquí tienes el desglose lógico por bloques:

1. La Memoria Inicial

C++



```
int ledActivo = -1;
```

El truco del -1: Como los arreglos en C++ empiezan a contar desde `0` (el primer LED es `lucesita[0]`), no podemos iniciar esta variable en cero, o la placa encendería la primera luz apenas se conecte. Le damos el valor `-1` para que signifique: "ningún botón ha sido presionado todavía".

2. Buscar quién fue presionado

C++



```
for (int i = 0; i < 8; i++) {
    if (botones[i].presionado) {
```

A diferencia del bucle con `auto&` que usamos arriba para actualizar el hardware, aquí usamos un bucle `for` tradicional con la letra `i` (que va de 0 a 7).

Hacemos esto porque **necesitamos el número del índice (`i`)**. Si detectamos que `botones[3]` fue presionado, usamos ese mismo `3` para afectar a `lucesita[3]`.

3. La lógica de reemplazo

C++



```
if (ledActivo != i) {
    if (ledActivo != -1) {
        lucesita[ledActivo].apagarLed();
    }
    ledActivo = i;
}
break;
```

Este es el corazón del cambio de luces, ocurre en fracciones de segundo:

`if (ledActivo != i)` : Revisa si el botón que acabas de presionar es **distinto** al que ya está activo. Si presionas el botón de la luz que ya está parpadeando, ignora el comando.

`if (ledActivo != -1)` : Antes de apagar la luz anterior, verifica que efectivamente hubiera una luz encendida. Esto previene un error fatal (intentar apagar `lucesita[-1]`, que no existe).

`lucesita[ledActivo].apagarLed()` : Apaga físicamente el LED que estaba oscilando hasta hace un microsegundo.

`ledActivo = i` : **Actualiza la memoria**. Si presionaste el botón 4, guarda el `4`.

`break;` : Aborta el bucle `for` inmediatamente. Si el usuario presiona dos botones a la vez con varios dedos, el sistema solo le hará caso al primero que encuentre en la lista, previniendo comportamientos erráticos.

4. La ejecución independiente

C++



```
if (ledActivo != -1) {  
    lucesita[ledActivo].oscilarLed();  
}
```

Fíjate que este bloque está **completamente fuera** de las comprobaciones de los botones.

El bucle `while (true)` llega a esta parte miles de veces por segundo. Como `ledActivo` tiene un número guardado (por ejemplo, el `4`), le envía el comando `oscilarLed()` a `lucesita[4]` de forma constante. Esto es lo que permite que el efecto visual de la oscilación se mantenga vivo en el tiempo sin que el usuario tenga el dedo pegado al botón.

porque en el punto 2, existe un `i++`

El `i++` es una abreviatura clásica en C++ (y muchos otros lenguajes) que significa **"súmale 1 al valor actual de i"**. Es exactamente lo mismo que escribir `i = i + 1`.

En el contexto de tu bucle `for`, el `i++` es el "motor" que hace que el sistema avance y revise el siguiente botón de la fila, en lugar de quedarse atascado en el primero.

Así es como trabajan juntas las tres partes del `for (int i = 0; i < 8; i++)`:

1. `int i = 0` **(El punto de partida)**: Crea la variable `i` y la inicializa en 0. En esta primera vuelta, el código revisa `botones[0]`.
2. `i < 8` **(El límite)**: Le dice al bucle que siga funcionando solo mientras `i` sea menor a 8.
3. `i++` **(El paso o avance)**: Cuando el código termina de revisar el estado del primer botón, el `i++` entra en acción y transforma el `0` en un `1`.

En la siguiente fracción de segundo, el bucle vuelve a empezar, pero ahora `i` vale 1, por lo que revisa `botones[1]`. Luego el `i++` lo sube a 2, revisa `botones[2]`, y así sucesivamente hasta llegar a `botones[7]`.

Cuando el `i++` sube el número a 8, la regla `i < 8` dice "hasta aquí llegamos", el bucle se rompe y el programa continúa con las luces.

Sin el `i++` , la variable `i` valdría `0` para siempre. El microcontrolador pasaría la eternidad revisando únicamente el `botones[0]` y jamás se enteraría si presionaste los demás.

revisa el código y ubica errores, no los corrigas.

```
// esto venia en wokwi
#include <stdio.h>
```

▼

Aquí están los errores de sintaxis y estructura ubicados en tu código:

Comentario conceptual erróneo: Escribiste `// mejor optar por un string` , pero la estructura que estás definiendo en la línea siguiente es un **arreglo** (array), no un string.

Falta de punto y coma (Línea 30): La declaración del arreglo

`Boton botones[] = { ... }` carece del punto y coma obligatorio (`;`) al final de la llave de cierre.

Uso incorrecto del constructor (Línea 44): Al crear el arreglo `Led lucesita[]` , estás intentando inicializar los objetos escribiendo `lucesita(16)` ,

`lucesita(17)` , etc. Debes llamar al constructor de la clase, que es `Led(16)` , `Led(17)` .

Falta de punto y coma (Línea 44): Al igual que con el arreglo de botones, la declaración del arreglo de LEDs no finaliza con su respectivo punto y coma (`;`).

Llaves mal cerradas y lógica atrapada: Nunca cerraste la llave del

`if (botones[i].presionado)` { antes de pasar a la parte que ejecuta la oscilación. Esto causa dos problemas graves en la cascada del código:

1. Toda la lógica independiente (la oscilación del LED, la lectura del potenciómetro, los `printf` y los `sleep_ms`) quedó atrapada *adentro* del `if` de presionar el botón y adentro del bucle `for` .
2. Debido a que pusiste un `break;` justo antes de intentar ejecutar la oscilación, el bucle se cortará inmediatamente y **todo** el código que está debajo del `break` se volverá inalcanzable (jamás se ejecutará).

Faltan llaves de cierre al final del documento: Al final de tu código solo hay dos llaves `}` . Por la forma en que anidaste los bloques, te están faltando llaves para

poder cerrar correctamente el bucle `for`, el `while (true)` y la función

```
int main()
```

como soluciono la oscilacion

```
void Led::oscilarLed() {
```



El primer problema que impide que tu luz oscile es un error de sintaxis con el signo igual: estás usando `==` en lugar de `=`.

En C++, `==` es una pregunta ("¿es encendido igual al nivel del gpio?"), mientras que un solo `=` es una orden ("asigna el nivel del gpio a la variable encendido").

Aquí tienes la corrección básica:

C++



```
void Led::oscilarLed() {
    // Un solo '=' para guardar el valor actual del pin en tu variable
    encendido = gpio_get_out_level(conexionLed);

    // ! invierte el valor (si era true, lo hace false)
    gpio_put(conexionLed, !encendido);

    sleep_ms(1000);
}
```

El problema de usar `sleep_ms(1000)`

Si dejas el código así, la luz va a parpadear, pero **tus botones dejarán de responder bien**. Como el método tiene un `sleep_ms(1000)`, cada vez que llames a `oscilarLed()`, toda tu Raspberry Pi Pico se congelará por un segundo completo. Durante ese segundo, el bucle `while (true)` no podrá leer si presionaste otro botón.

Para arreglar esto, necesitas usar el concepto de **"Blink without delay"** (parpadear sin pausas), usando el reloj interno de la placa para medir el tiempo.

La solución definitiva (No bloqueante)

1. En tu archivo `Led.h`, debes agregar una nueva variable para recordar cuándo fue la última vez que la luz cambió de estado:

C++



```
class Led {  
    public:  
        // ... tus otras variables ...  
        uint32_t ultimoCambio = 0; // Agrega esta línea para guardar el tiempo  
        // ...  
};
```

2. En tu archivo `Led.cpp`, reescribe tu método así:

C++



```
void Led::oscilarLed() {  
    // Obtiene el tiempo actual en milisegundos desde que encendiste la placa  
    uint32_t tiempoActual = to_ms_since_boot(get_absolute_time());  
  
    // Si la diferencia entre el tiempo actual y el último cambio es mayor a 500ms  
    if (tiempoActual - ultimoCambio >= 500) {  
  
        // Cambiamos el estado del LED  
        encendido = gpio_get_out_level(conexionLed);  
        gpio_put(conexionLed, !encendido);  
  
        // Guardamos el nuevo tiempo para la próxima vez  
        ultimoCambio = tiempoActual;  
    }  
}
```

De esta manera, el método `oscilarLed()` entra, revisa la hora y, si no ha pasado medio segundo, sale inmediatamente. Esto permite que tu programa regrese rapidísimo al `while (true)` a seguir leyendo los botones en tiempo real, mientras la luz parpadea a su propio ritmo.